Reinforcement Learning with the Card Game Scoundrel

Colby Cox

Due: April 17, 2026

1 Introduction

Scoundrel is a game whose probability to win is largely influenced by the initial shuffle of the deck. The player can be swarmed by monsters and immediately lose, or swamped by helpful items that can not be used to their fullest. Winning Scoundrel sometimes comes down to a advantageously shuffled spread of items and monsters, but how much is winning within the player's control?

This project originally sought to find a lower bound for the win rate by using reinforcement learning and comparing to a human player. However, after extensive testing, I have determined that the conditions of the game Scoundrel make generalizing it for any set of shuffled cards make learning very difficult. Some other examples of successful implementations of reinforcement learning include Chess and Atari games like Breakout and Pong [4]. Chess is highly complex, with an upper bound for the number of legal game states being about 7.7×10^{45} [7] . Compare this to Scoundrel, which is a deck of 44 cards and thus has 44!, about 2.7×10^{54}, possible game states at the beginning of the game. Without a highly specific implementation of reinforcement learning capable of generalizing the relationship of the cards for the computer, it makes solving Scoundrel for any deck highly complex. Meanwhile Atari games are much simpler, and have a much smaller number of potential game states than Scoundrel, so their implementation can be much more general. As such, I've reduced the scope to asking if we can find a set of decisions that can beat a given deck without human involvement using reinforecement learning.

2 The Rules

Scoundrel was created by Zach Gage and Kurt Bieg, and their rule set can be found at
http://www.stfj.net/art/2011/Scoundrel.pdf [3]

2.1 Setup

To play Scoundrel with a deck of cards, first remove the jokers, red face cards, and red aces, as they're not used, and shuffle the remaining cards. The remaining cards make up the dungeon, and our goal is to move every card from the dungeon to the discard pile. The player starts with 20 health, which can be tracked with a piece of paper and pen or a d20.

2.2 Cards

The remaining 44 cards are monsters, weapons, or health potions, depending on their suit. The value of each non-face card is indicated by its number, while the jack is worth 11, the queen 12, the king 13, and the ace 14.

- The 26 Clubs and Spades are monsters. Below is the process for damage calculation.
    1. If the player does not have a weapon, then the player's health is reduced by the value of the monster.
    2. If the player has a weapon that has not attacked a monster, then the player's health is reduced by the value of the monster minus the value of the weapon, with a minumum of 0 damage taken.

3. If the player has a weapon that has attacked a monster and the current monster's value is less than the previous monster's value, then the player's health is reduced by the value of the monster minus the value of the weapon, with a minumum of 0 damage taken.
4. If the player has a weapon that has attacked a monster and the current monster's value is greater than the previous monster's value, then the player's health is reduced by the value of the current monster.

- The 9 Diamond cards are weapons. When a new weapon is selected, the previous is discarded.
- The 9 Heart cards are health potions, which increase your health by the value of the card up to 20.

2.3 Game Loop

The game starts by drawing four cards. The player can decide each turn at the start of every encounter to run from the room or stay. If the player runs, the four cards are placed on the bottom of the deck in a random order. If they stay, the player must pick three of the four cards one at a time. Once three cards have been selected, the player draws three more cards. This process repeats until either the player has 0 health, which means they lost, or the player has selected every card in the dungeon, meaning they win. After a game has finished, the player's score can be calculated based on their performance:

- If the player lost, then the player's score is the sum of all the remaining monsters' values in the dungeon.
- If the player won, then the player's score is their positive health value.

2.4 Rule Changes

Compared to the original, this version has a few differences to make implementing it with reinforcement learning with it simpler without too much deviation.

1. In the original, attacking barehanded is allowed, meaning the player can choose to not use their weapon to not add the monster to the player's weapon. Doing this would increase the action space from 5 to 7, drastically increasing the complexity, so instead a monster is added to the weapon whenever the player attacks with the weapon.
2. If a heart card is the last item picked up and the player can heal over 20 health, the health is added to the player's final score. Scoundrel is tough enough that unless the deck was incredibly well shuffled, this rule would never come into effect.
3. The player can only pick up one health potion per room. This rule would have the same effect as the model being able to pick running from a room multiple times while not actually doing it, so it's been excluded.

3 Methodology

3.1 Background

This introduction to reinforcement learning is paraphrased from *Data-Driven Science and Engineering: Machine Learning, Dynamical Systems and Control* [2]. For our model to make decisions, we need to make a policy table that, given the environment or state s and the action a, gives a future reward value r. The goal is then to maximize the future reward using the policy table that gives maximized rewards, defined as $\pi(s, a)$.

$$\pi(s, a) = \Pr(a = a | s = s)$$

For most problems, representing and learning this policy becomes prohibilitely expensive, and π must instead be represented as an approximate function parameterized by a lower-dimensional vector θ

$$\pi(s, a) \approx \pi(s, a, \theta)$$

which is often denoted $\pi_\theta(s, a)$.

Given a policy π, we next define a value function that quantifies the desireability of being in a given state:

$$V_\pi(s) = \mathbb{E}\left(\sum_{k=0}^{\infty} \gamma^k r_k \,\middle|\, s_0 = s\right)$$

where $\mathbb{E}$ is the expected future reward given a policy π and subject to a discount rate γ. Since the value function uses γ^k, future rewards are less valuable than current rewards. The subscript π is omitted from the value function, in which case we refer to the value function for the best possible policy:

$$V(s) = \max_\pi \mathbb{E}\left(\sum_{k=0}^{\infty} \gamma^k r_k | s_0 = s\right)$$

By letting s' be s_{k+1}, the next state, the value function can be written as

$$V(s) = \max_\pi \mathbb{E}\left(r_0 + \sum_{k=1}^{\infty} \gamma^k r_k | s_1 = s'\right)$$

which implies that

$$V(s) = \max_\pi \mathbb{E}(r_0 + \gamma V(s'))$$

Given the value function, it is possible to extract the optimal policy as

$$\pi = \arg\max_\pi \mathbb{E}(r_0 + \gamma V(s'))$$

3.2 Proximal Policy Optimization

This explanation of proximal policy optimization (PPO) is paraphrased from Open AI's paper on the subject from 2017 [6]. The goal of PPO is to maximize the following objective function:

$$L_t^{CLIP+VF+S}(\theta) = \hat{\mathbb{E}}_t[L_t^{CLIP}(\theta) - c_1 L_t^{VF}(\theta) - c_2 S[\pi_\theta](s_t)]$$

where c_1 is the VF coefficient, c_2 is the entropy coefficient, S denotes an entropy bonus, and L_t^{VF} is the squared-error loss.

$$L_t^{VF} = (V_\theta(s_t) - V_t^{targ})^2$$

The clip function L_t^{CLIP} limits the probability ratio between new and old policies, which prevents excessively large policy updates that could destabilize learning.

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t\left[\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t)\right]$$

The advantage function $\hat{A}_t$ guides the policy update to increase the probability of better actions.

$$\hat{A}_t = \delta_t + (\gamma\lambda)\delta_{t+t} + ... + (\gamma\delta)^{T-t+1}\delta_{T-1},$$
$$\text{where } \delta_t = r_t + \gamma V(s_t + 1) - V(s_t)$$

In the actual implementation, the following is algorithm is used to define PPO.

Algorithm PPO, Actor-Critic Style

```
for iteration = 1, 2, ... do
    for actor = 1, 2, ... N do
        Run policy π_θ_old in environment for T timesteps
        Compute advantage estimates for Â_1, ..., Â_T
    end for
    Optimize surrogate L wrt θ with K epochs and minibatch size M ≤ NT
    θ_old ← θ
end for
```

3.3 Hyperparameters

Below is each hyperparameter and its meaning:

- Horizon (T): Number of timesteps to run environment on old policy.
- Number of Epochs: How many times each batch of data is used for policy updates.
- Minibatch size: Number of samples per update.
- Discount (γ): How much future rewards are valued compared to immediate rewards.
- GAE Parameter (λ): Uses Generalized Advantage Estimation to balance bias and variance of advantage estimation.
- Number of actors: Number of simulated environment contributing in parallel.
- Clipping parameter (ϵ): The amount the new policy can deviate from the old policy.
- VF. coefficient (c_1): Weight given to the critic loss in the total objective.
- Entropy coefficient (c_2): Encourages exploration by penalizing low entropy.

These, along with the reward function, are the main parts we must determine to appropriately train our model.

4 Experiment Design

There are two halves to the experiment, one in Python and the other in Godot. Python manages the reinforcement learning algorithm and the storage of the model. Godot manages the game, what the model can observe, the actions the model can take, and the reward value. The plugin Godot RL Agents [1] along with Stable Baselines 3 [5] are used to connect Godot and Python and implement the algorithm. The game is written exactly as described in the rules section, while how the other elements are managed is written below.

4.1 Observations

The model is given a 8 by 1 matrix of integers as its observation. The first four values are the current health's value, the current weapon's value, the monster that is on the weapon's value, and a boolean of whether or not the run button is disabled. The following four represent the status of each card slot, with a 0 if there is no card, a 1 if the card is a monster, a 2 if the card is a weapon, and a 3 if the card is a health potion.

An additional four values of each card's value could have been used, but they increase the observation space dramatically and in testing, significantly slowed down the learning process. The values could also be normalized, but when this was applied, the model reacted less predictably and did not learn as quickly.

4.2 Actions

The model can choose an integer from 0 to 4 to make its choice. If it chooses 0, then the player runs. If it doesn't, then it chooses the card that matches the position of the integer chosen, 1, 2, 3, or 4. It is implemented like this since the player can only choose one action per turn.

4.3 Reward

For the reward function, we want to represent the algorithm making decisions that represent how it is getting closer, further away, or is stagnant in getting to the goal. For example, in Snake, the reward function could be the change in distance from the snake's head to the food. For this implementation, I have assigned a reward of the sum of all the monster cards' values in the previous iteration minus the sum of all monster

cards' values in the current iteration. This represents how when a monster is removed from the hand, the value of that monster is added as the reward for that step.

The model can choose to run while running is unavailable or choose a card slot that doesn't have a card. This can be refined by the model over time since those choices don't add anything to the reward value, and those choices could be removed from the choice list after the fact.

4.4 Hyperparameters

The hyperparameters for Stable Baseline's implementation of PPO selected were:

- ent_coef (c_2) = 0.1
- n_steps (T) = 1024
- learning_rate = 0.0003

The hyperparameters not mentioned were left at there defaults according to Stable Baseline's PPO implementation. There were also the hyperparameters of the Godot half, which were:

- timesteps = 10,000,000
- n_parallel = 4
- speedup = 8

5 Results

In the first successful trial with the previously discussed parameters, the model had won about 1,600 games out of 180,000 after 10,002,432 timesteps and 4 and a half hours of training. Below is the final output given. Note that the trials, wins and score output is for each individual process, while the final output is for the overall model, so the overall number of trials and wins can be found by multiplying by 4.

```
Trials: 45066  Wins: 406  Score: -26
Trials: 45020  Wins: 400  Score: 0
Trials: 44986  Wins: 429  Score: 3
Trials: 44963  Wins: 430  Score: -5
Trials: 44964  Wins: 430  Score: -23
Trials: 45021  Wins: 401  Score: 3
Trials: 45067  Wins: 407  Score: 3
Trials: 44987  Wins: 429  Score: 0
Trials: 44988  Wins: 429  Score: -61
Trials: 44965  Wins: 431  Score: 3
Trials: 45022  Wins: 401  Score: -7
Trials: 45068  Wins: 408  Score: 3
Trials: 44966  Wins: 431  Score: -140
Trials: 44967  Wins: 431  Score: -105
Trials: 44989  Wins: 429  Score: -26
Trials: 45023  Wins: 402  Score: 3
Trials: 45069  Wins: 408  Score: -26
Trials: 44968  Wins: 432  Score: 3
Trials: 45024  Wins: 402  Score: -9
Trials: 44990  Wins: 429  Score: 0
Trials: 45070  Wins: 409  Score: 3
Trials: 44969  Wins: 433  Score: 3
Trials: 45025  Wins: 402  Score: -46
Trials: 44991  Wins: 429  Score: -26
Trials: 45071  Wins: 410  Score: 3
-----------------------------------------
| rollout/                |             |
|    ep_len_mean          | 60.3        |
|    ep_rew_mean          | 181         |
| time/                   |             |
|    fps                  | 615         |
|    iterations           | 2442        |
|    time_elapsed         | 16261       |
|    total_timesteps      | 10002432    |
| train/                  |             |
|    approx_kl            | 0.0074769985 |
|    clip_fraction        | 0.0766      |
|    clip_range           | 0.2         |
|    entropy_loss         | -0.867      |
|    explained_variance   | 0.97        |
|    learning_rate        | 0.0003      |
|    loss                 | 33.2        |
|    n_updates            | 24410       |
|    policy_gradient_loss | -0.0109     |
|    value_loss           | 53.3        |
-----------------------------------------
```

(a) First Success

```
Trials: 41532  Wins: 24694  Score: 6
Trials: 41529  Wins: 24888  Score: 2
Trials: 41458  Wins: 24973  Score: 10
Trials: 41533  Wins: 24694  Score: -102
Trials: 41530  Wins: 24888  Score: -59
Trials: 41572  Wins: 24649  Score: 6
Trials: 41459  Wins: 24974  Score: 7
Trials: 41534  Wins: 24694  Score: -59
Trials: 41531  Wins: 24889  Score: 10
Trials: 41573  Wins: 24650  Score: 6
Trials: 41460  Wins: 24975  Score: 7
Trials: 41574  Wins: 24650  Score: -177
Trials: 41535  Wins: 24695  Score: 6
Trials: 41532  Wins: 24890  Score: 3
Trials: 41575  Wins: 24651  Score: 7
Trials: 41536  Wins: 24695  Score: -6
Trials: 41461  Wins: 24976  Score: 7
Trials: 41533  Wins: 24890  Score: -51
Trials: 41537  Wins: 24696  Score: 3
Trials: 41576  Wins: 24652  Score: 3
Trials: 41462  Wins: 24977  Score: 9
Trials: 41534  Wins: 24891  Score: 6
Trials: 41577  Wins: 24653  Score: 10
-----------------------------------------
| rollout/                |             |
|    ep_len_mean          | 61          |
|    ep_rew_mean          | 190         |
| time/                   |             |
|    fps                  | 248         |
|    iterations           | 2442        |
|    time_elapsed         | 40324       |
|    total_timesteps      | 10002432    |
| train/                  |             |
|    approx_kl            | 0.013040384 |
|    clip_fraction        | 0.109       |
|    clip_range           | 0.2         |
|    entropy_loss         | -0.93       |
|    explained_variance   | 0.939       |
|    learning_rate        | 0.0003      |
|    loss                 | 24.6        |
|    n_updates            | 24410       |
|    policy_gradient_loss | -0.0133     |
|    value_loss           | 126         |
-----------------------------------------
```

(b) Same Deck Reran

We can see that the second time, the model got lucky and found a winning combination of choices much earlier. As I was watching it, it happened at about 8,000 trials total. It also finds a much better final score of 10, so we can run many trials, but may not find the best solution considering how many possible combinations there are.

The second test was another success with a new deck, but I had been checking to see if the deck was beatable, and the previous two were. With the third trial, I wished to see if the model could find a solution where I couldn't, so it was given a deck I couldn't beat after a few attempts. Below are the second and third tests:

```
Trials: 43274  Wins: 4450  Score: -5
Trials: 43311  Wins: 4393  Score: -132
Trials: 43330  Wins: 4397  Score: -22
Trials: 43377  Wins: 4342  Score: -39
Trials: 43275  Wins: 4450  Score: -22
Trials: 43312  Wins: 4393  Score: 0
Trials: 43331  Wins: 4397  Score: -22
Trials: 43378  Wins: 4342  Score: -11
Trials: 43313  Wins: 4393  Score: -58
Trials: 43276  Wins: 4451  Score: 3
Trials: 43379  Wins: 4342  Score: -11
Trials: 43314  Wins: 4394  Score: 3
Trials: 43277  Wins: 4452  Score: 2
Trials: 43332  Wins: 4397  Score: -15
Trials: 43380  Wins: 4343  Score: 3
Trials: 43278  Wins: 4452  Score: -17
Trials: 43315  Wins: 4395  Score: 3
Trials: 43333  Wins: 4397  Score: -4
Trials: 43316  Wins: 4396  Score: 3
Trials: 43381  Wins: 4344  Score: 3
Trials: 43279  Wins: 4453  Score: 3
Trials: 43334  Wins: 4397  Score: -8
Trials: 43317  Wins: 4396  Score: -27
Trials: 43280  Wins: 4453  Score: -69
Trials: 43382  Wins: 4344  Score: 0
-----------------------------------------
| rollout/                |            |
|    ep_len_mean          | 66.7       |
|    ep_rew_mean          | 179        |
| time/                   |            |
|    fps                  | 739        |
|    iterations           | 2442       |
|    time_elapsed         | 13523      |
|    total_timesteps      | 10002432   |
| train/                  |            |
|    approx_kl            | 0.012174594 |
|    clip_fraction        | 0.0936     |
|    clip_range           | 0.2        |
|    entropy_loss         | -0.987     |
|    explained_variance   | 0.921      |
|    learning_rate        | 0.0003     |
|    loss                 | 51.5       |
|    n_updates            | 24410      |
|    policy_gradient_loss | -0.0149    |
|    value_loss           | 88.1       |
-----------------------------------------
```

(a) Second Test

```
Trials: 45713     Wins: 1     Score: -8
Trials: 45733     Wins: 1     Score: -36
Trials: 45748     Wins: 0     Score: -11
Trials: 45800     Wins: 0     Score: -9
Trials: 45749     Wins: 0     Score: -85
Trials: 45714     Wins: 1     Score: -10
Trials: 45734     Wins: 1     Score: -8
Trials: 45801     Wins: 0     Score: -8
Trials: 45735     Wins: 1     Score: -20
Trials: 45750     Wins: 0     Score: -9
Trials: 45715     Wins: 1     Score: -8
Trials: 45802     Wins: 0     Score: -9
Trials: 45803     Wins: 0     Score: -170
Trials: 45804     Wins: 0     Score: -123
Trials: 45751     Wins: 0     Score: -8
Trials: 45716     Wins: 1     Score: -8
Trials: 45736     Wins: 1     Score: -8
Trials: 45805     Wins: 0     Score: -9
Trials: 45752     Wins: 0     Score: -11
Trials: 45717     Wins: 1     Score: -8
Trials: 45737     Wins: 1     Score: -8
Trials: 45806     Wins: 0     Score: -11
Trials: 45753     Wins: 0     Score: -8
Trials: 45718     Wins: 1     Score: -8
Trials: 45738     Wins: 1     Score: -8
-----------------------------------------
| rollout/                |            |
|    ep_len_mean          | 58.6       |
|    ep_rew_mean          | 177        |
| time/                   |            |
|    fps                  | 597        |
|    iterations           | 2442       |
|    time_elapsed         | 16743      |
|    total_timesteps      | 10002432   |
| train/                  |            |
|    approx_kl            | 0.015231494 |
|    clip_fraction        | 0.0951     |
|    clip_range           | 0.2        |
|    entropy_loss         | -0.885     |
|    explained_variance   | 0.976      |
|    learning_rate        | 0.0003     |
|    loss                 | 12.7       |
|    n_updates            | 24410      |
|    policy_gradient_loss | -0.0103    |
|    value_loss           | 43.6       |
-----------------------------------------
```

(b) Third Test

The third test was unsuccessful in finding a winning solution, but not necessarily because the deck is unbeatable. It only scored a couple of wins, but the first was around 80,000 total trials, so about halfway through. It then converges on -8, meaning that a winning solution was possible but may have had many runs involved which would have lowered the reward. To alleviate this problem of losing the best solution, I implemented a program to save the best solution once the current best score has been beaten.

A fourth test was ran with a new deck and a final max score of zero. Its choice list, along with all of the trials' models are located in the Github Repository `https://github.com/ColbysPrograms/ScoundrelRL`

6 Future Considerations

Potential future considerations could include:

- Hyperparameter Tuning. The hyperparameters were changed manually based on the model's performance, like increasing the entropy coefficient if the model wasn't exploring enough.
- Different Algorithms. Monte Carlo Tree Search would have been a better choice for the objective of finding a winning list of choices for an individual shuffle of the deck since it does not need to repeat the same path to ensure repeatability like PPO does.
- Optimizing the observation space. Currently the observation space does not include the values of each card. Doing so drastically increases the potential observation spaces from $20 \cdot 9 \cdot 13 \cdot 2 \cdot 4(4) = 74,880$ to $20 \cdot 9 \cdot 13 \cdot 2 \cdot 4(4) \cdot 4(13) = 3,893,760$, so it was not included for the time and computational constraints in this project, but simplifying this relationship would be highly valuable to finding the optimal win rate for Scoundrel.
- Interpreting the decisions of the model. For this experiment, it searches for the best possible path to win the game. This could be observed and interpretted to find better strategies in balancing the risk of running from a room.
- Using exclusively Python. Much of the information for reinforcement learning and online implementations are in Python, using Gymnasium and Pygame to create the environment and using an algorithm implementation like Stable Baselines 3's implementation of PPO.

7 Conclusion

This project is successful in its secondary goal of finding paths that can beat the game Scoundrel, but only with sufficient time and resources. There is plenty that could be done to clean up and make a much better model, such as using a different algorithm or simplifying the observation space, but it's very similar to the current state of reinforcement learning as a whole, with constant updates and new ideas. With more fine tuning, this model could eventually be used to solve Scoundrel in general, like the original mission statement of this project.

This project's Github Repository is hosted at
`https://github.com/ColbysPrograms/ScoundrelRL`

8 References

[1] Edward Beeching. *Godot RL Agents*. original-date: 2021-07-01T14:04:11Z. Feb. 2025. URL: `https://github.com/edbeeching/godot_rl_agents` (visited on 04/19/2026).

[2] Steven L. Brunton and J. Nathan Kutz. *Data-driven science and engineering: machine learning, dynamical systems, and control*. eng. Cambridge New York, NY Port Melbourne New Delhi Singapore: Cambridge University Press, 2019. ISBN: 9781108422093.

[3] Zach Gage and Kurt Bieg. *Scoundrel - Version 1.0*. Aug. 2011. URL: `http://www.stfj.net/art/2011/Scoundrel.pdf`.

[4] Volodymyr Mnih et al. *Playing Atari with Deep Reinforcement Learning*. arXiv:1312.5602. Dec. 2013. DOI: `10.48550/arXiv.1312.5602`. URL: `http://arxiv.org/abs/1312.5602` (visited on 04/18/2026).

[5] Antonin Raffin et al. "Stable-Baselines3: Reliable Reinforcement Learning Implementations". In: *Journal of Machine Learning Research* 22.268 (2021), pp. 1–8. URL: `http://jmlr.org/papers/v22/20-1364.html`.

[6] John Schulman et al. *Proximal Policy Optimization Algorithms*. arXiv:1707.06347. Aug. 2017. DOI: `10.48550/arXiv.1707.06347`. URL: `http://arxiv.org/abs/1707.06347` (visited on 04/18/2026).

[7] John Tromp. *Chess Position Ranking*. original-date: 2021-07-19T13:41:33Z. Aug. 2021. URL: `https://github.com/tromp/ChessPositionRanking` (visited on 04/18/2026).